第 02 章

资料解析、字符分块与元数据

从真实资料上传出发，读懂文件怎样变成可追溯、可重复处理的知识片段。

这一章完成后，你能够

- 说明 bytes、Document 和 DocumentChunk 的输入输出区别。
- 读懂多格式解析、Markdown 标题路径和递归字符分块。
- 解释 800/120 的单位、稳定 ID、文本哈希与页码的用途。
- 运行离线实验，识别分块边界和长资料覆盖限制。

学习前提

Python 函数、列表、字典与 UUID；第 1 章的资料上传和任务处理主链路；已准备好的 apps/api/.venv 学习环境

代码基准：8aa0ba5 + 本轮名称与分页修改。Python 3.12；LangChain 1.4.3；LangGraph 1.2.12。

正文采用场景、名词、步骤、源码、实验和练习的教学顺序。截图注明数据来源；教学简化与生产实现分别标记。

本章目录

2.1 业务问题：上传成功以后，资料真的能用于学习吗? 3

2.2 名词解析：从 bytes 到 Document，再到业务片段 4

2.3 多格式解析：统一接口，保留各自限制 5

2.4 Markdown 标题路径：为什么切出来还要知道属于哪一节? 7

2.5 递归字符分块：800 和 120 到底控制什么? 8

2.6 片段身份与元数据：重复处理怎样保持可追溯? 10

2.7 重新处理与失效：为什么更新材料会影响题目? 11

2.8 完整离线实验：自己观察标题、块边界和稳定 ID 12

2.9 本章回顾：分块完成，为什么还不能保证全文覆盖? 13

2.10 综合练习与参考思路 14

2.11 源码阅读清单与下一步 15

2.1 业务问题：上传成功以后，资料真的能用于学习吗？

打开项目知识库页面，上传一份讲义后，你最关心的是它能否用于出题和问答。文件保存成功只说明原始数据有了存放位置；解析器还要读出文字，分块器还要建立片段，索引任务还要完成向量写入。页面上的处理状态，就是这条数据准备链路的外部表现。

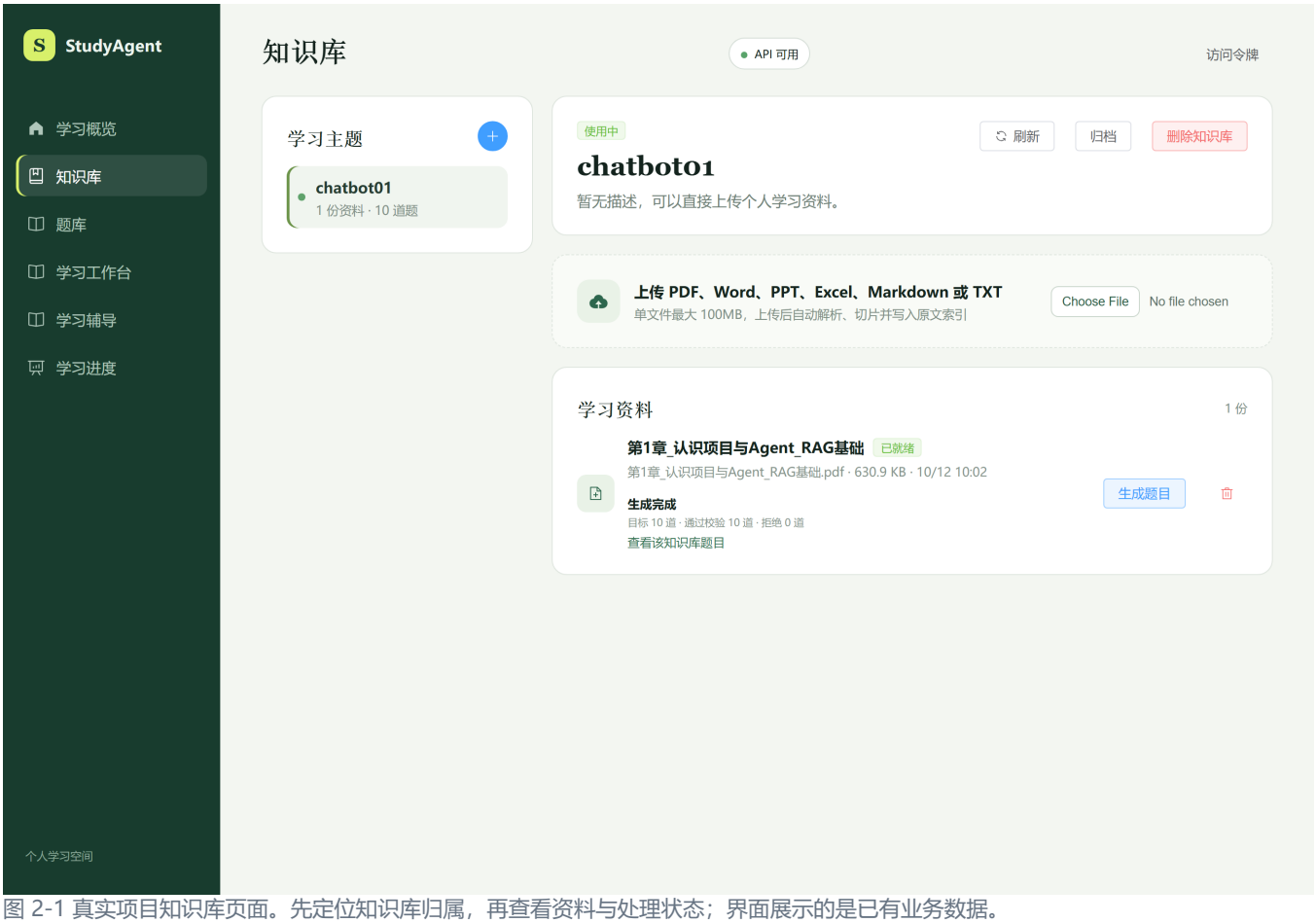


图 2-1 真实项目知识库页面。先定位知识库归属，再查看资料与处理状态；界面展示的是已有业务数据。

当前真实项目页面 / 只读现有业务数据 / 2026-10-12。界面数字只代表截图时的可见记录；没有开始学习或提交出题。



图 2-2 资料准备的实际阶段。前三步是本章重点，Embedding 与检索在第 3 章展开。

入口会同时创建 `Material` 和 `IngestionJob`。前者描述资料，后者描述一次处理过程。`CeleryIngestionDispatcher.dispatch` 依次串起解析、索引、完成三个任务。这里是固定顺序的后台工作流，程序已经决定执行步骤，不依赖模型判断下一步应调用哪个工具。

观察对象	当前值或阶段	应该怎样理解
Material.parse_status	pending / parsing / ready / failed	资料是否具备被后续链路使用的条件
IngestionJob.stage	uploaded → parsing → chunking → embedding → completed	本次后台任务走到哪个阶段
progress	10、60、70、90、100 等预设值	阶段性进度提示，不是实时计算的耗时百分比
parser_version	完成后为 p2-v1	内部解析版本标签，不代表 LangChain 软件版本

表 2-1 资料状态与任务状态的区别。任务阶段比界面的一句“处理中”更有利于排查。

学习时的观察顺序

先看任务是否 completed，再看资料是否 ready，最后检查是否存在片段。只看到上传列表中的文件名，还不足以证明解析与索引完成。

2.2 名词解析：从 bytes 到 Document，再到业务片段

原始文件是 `bytes`，也就是字节序列。解析器把字节转换成 `Document`：其中 `page_content` 保存文字，`metadata` 保存来源说明。一个 PDF 通常产生多个按页组织的 Document；Markdown 则可能按标题形成多个 Document。因此，Document 在这里是一段统一表示的数据，不必等同于一个完整文件。

名词 / 对象	核心内容	本项目的职责
bytes / 文件字节	编码后的原始文件	交给相应格式解析器读取
Document / 文档对象	page_content + metadata	统一多格式解析结果，暂不要求数据库身份
metadata / 元数据	来源、页码、标题路径等	让文字具备来源和结构信息
DocumentChunk / 业务片段	ID、归属、序号、正文、哈希等	进入数据库，供索引、出题与引用使用

表 2-2 相似名称代表不同层次。Document 是框架对象，DocumentChunk 是项目领域对象。



图 2-3 概念示意：一页可以产生多个块，它们仍保留第 3 页来源。本图不是当前截图资料的实际块数。

`_document` 会去掉每行末尾空白，再去掉整个文本两端空白。空文本返回 `None`，由各格式解析器忽略。它没有做深度内容清洗，例如识别页眉页脚、修复表格或判断事实是否正确。学习时先分清“文字被读出来”和“文字被正确整理”这两个目标，后续才能准确定位问题。

metadata 不是模型答案

`heading_path=['网络','三次握手']` 描述这段文字属于哪里，不说明它是否正确。元数据帮助过滤与追溯；内容可靠性还需要来源检查和人工判断。

2.3 多格式解析：统一接口，保留各自限制

`parse_document(content, filename)` 接收完整文件字节和文件名，按文件后缀找到解析函数，返回 `list[Document]`。接口统一后，后续切分器不需要知道输入曾经是 PDF 还是 Excel。但不同格式的读取能力仍由各自实现决定，统一类型并不会自动带来统一的版式理解能力。

源码节选：根据后缀分派解析器

定位：apps/api/study_agent/ingestion/parsers.py，第 140-160 行。

```
PARSERS: dict[str, Callable[[bytes, str], list[Document]]] = {
    ".txt": _parse_text,
    ".md": _parse_markdown,
    ".markdown": _parse_markdown,
    ".pdf": _parse_pdf,
    ".docx": _parse_docx,
    ".pptx": _parse_pptx,
    ".xlsx": _parse_xlsx,
}


# 根据文件扩展名选择解析器，统一返回带来源信息的 Document 列表。
def parse_document(content: bytes, filename: str) -> list[Document]:
```

代码续

```
153     suffix = Path(filename).suffix.lower()
154     parser = PARSERS.get(suffix)
155     if parser is None:
156         raise ValueError(f"unsupported parser for suffix: {suffix}")
157     documents = parser(content, filename)
158     if not documents:
159         raise ValueError("document contains no extractable text")
160     return documents
```

PARSERS 是“后缀 → 函数”的字典，Callable 描述函数可以被调用。

content 是 bytes；filename 用来选择解析器；返回非空 Document 列表。

不支持的后缀和没有可提取文字的文件，分别触发明确的 ValueError。

格式	读取与组织方式	学习时要确认的边界
TXT	utf-8-sig 解码，形成一个 Document	其他编码不会自动识别；空文本被拒绝
Markdown	按 1～6 级 # 标题组织正文	标题保存在 metadata，解析不是完整 Markdown 语法树
PDF	pypdf 逐页 extract_text	有文字层才能提取；没有接入 OCR
DOCX	遍历段落，识别 Heading 样式	当前循环没有读取表格、图片与页码
PPTX	读取带 text 属性的形状	幻灯片编号用作页码；图片文字没有识别
XLSX	按工作表读取单元格，使用制表符拼接	data_only 读取已有公式缓存，不负责计算公式

表 2-3 按当前源码填写的能力表。课程提到的 OCR 或复杂版面工具属于可扩展方案。

源码节选：PDF 逐页加载

定位：apps/api/study_agent/ingestion/parsers.py，第 56-68 行。

```
56  def _parse_pdf(content: bytes, filename: str) -> list[Document]:
57      documents: list[Document] = []
58      for page_number, page in enumerate(PdfReader(BytesIO(content)).pages, start=1):
59          document = _document(
60              page.extract_text() or "",
61              source=filename,
62              page_start=page_number,
63              page_end=page_number,
64              heading_path=[],
65          )
66          if document:
67              documents.append(document)
68      return documents
```

BytesIO 把内存中的 bytes 包装成文件一样的对象，避免先落地一个临时文件。

enumerate(..., start=1) 使页码从 1 开始，page_start 与 page_end 都取当前页。

extract_text 返回空值时使用空字符串；没有文字的页不会生成 Document。

例如，一份 PDF 的第 2 页是纯图片，第 3 页有可提取文字，解析结果可以只有第 1 页和第 3 页，页码仍应保持原始编号。看到 Document 数量少于 PDF 页数时，应先检查页面是否有文字层。若原文有复杂表格，还应核对列顺序，不能把“没有抛异常”直接当作解析质量合格。



图 2-4 Vue 资料卡片配合隔离 TCP 教学数据：ready、目标 10、通过 10、拒绝 0；资料状态与题目审核是不同阶段。

真实 Vue 页面 / 隔离 HTTP fixture 教学数据 / 2026-10-09；非真实模型出题记录。固定教学完成态：目标 10、通过 10、拒绝 0；与已保存的合成验收数字同形，但本图不读取真实测试库

2.4 Markdown 标题路径：为什么切出来还要知道属于哪一节？

设想讲义先讲 TCP，再讲 Redis。如果只保存正文“它用于建立连接”，检索结果可能丢失“它”所指的对象。Markdown 解析器会把标题层级放入 `heading_path`。读到新标题时，先把已有正文写成 Document，再调整标题栈。这样正文和它所属的章节可以同时进入后续分块流程。

源码节选：切换标题前先保存旧正文

定位：apps/api/study_agent/ingestion/parsers.py，第 32-50 行。

```
32      def flush() -> None:
33          document = _document("\n".join(buffer), source=filename, heading_path=list(headings))
34          if document:
35              documents.append(document)
36          buffer.clear()
37
38      for line in text.splitlines():
39          stripped = line.lstrip()
40          if stripped.startswith("#"):
41              marker = stripped.split(maxsplit=1)[0]
42              if set(marker) == {"#"} and 1 <= len(marker) <= 6:
43                  flush()
44                  title = stripped[len(marker) :].strip()
45                  level = len(marker)
46                  headings[:] = headings[: level - 1]
47                  if title:
48                      headings.append(title)
49                  continue
50          buffer.append(line)
```

buffer 暂存正文行，headings 暂存当前标题层级，flush 是函数内部的小函数。

list(headings) 创建当前路径的副本，避免后面修改标题栈时改变已经保存的路径。

遇到 ## 三次握手时，保留一级标题“网络”，再补二级标题“三次握手”。

读到的内容	heading_path	正文归属
# 网络	['网络']	仅更新标题，标题行本身不进入正文
TCP 提供可靠传输。	['网络']	保存为网络节下的正文
## 三次握手	['网络','三次握手']	先保存上一段，再切换到子节
## 四次挥手	['网络','四次挥手']	同级标题替换上一项

表 2-4 标题栈的逐步变化。标题路径是列表，字符串展示时可以用斜杠连接。

标题不会作为正文重复存储，而会作为 metadata 附着在 Document 上。这能减少正文重复，也要求后续链路在需要时显式带上标题。例如出题上下文会打印标题路径，Milvus 也会存储它。当前解析器以行首 # 判断标题，没有处理代码围栏的独立状态；包含复杂 Markdown 代码块时，应专门检查是否误识别。

练习：标题也是可观察数据

把实验中的“## 三次握手”改成“## 连接建立”，先比较正文，再比较 heading_path。只修改标题不一定改变片段文本哈希；稳定 ID 不能替代对元数据变化的检查。

2.5 递归字符分块：800 和 120 到底控制什么？

`build_chunks` 接收解析后的 Document 列表、材料 ID、块大小和重叠大小，返回业务 `DocumentChunk` 列表。项目使用 `RecursiveCharacterTextSplitter`，没有指定 Token 计数器，因此长度按字符计算。配置里的 800 和 120，分别是目标块大小与目标重叠字符数，不能改写成“每块 800 Token”。

源码节选：递归分隔与稳定 ID 的建立

定位：apps/api/study_agent/ingestion/chunking.py，第 29-43 行。

```
29      splitter = RecursiveCharacterTextSplitter(
30          chunk_size=chunk_size,
31          chunk_overlap=chunk_overlap,
32          add_start_index=True,
33          separators=["\n\n", "\n", "。", "! ", "? ", ". ", " ", ""],
34      )
35      split_documents = splitter.split_documents(documents)
36      chunks: list[DocumentChunk] = []
37      for index, document in enumerate(split_documents):
38          content = document.page_content.strip()
39          if not content:
40              continue
41          content_hash = hashlib.sha256(content.encode("utf-8")).hexdigest()
42          chunk_id = uuid5(CHUNK_NAMESPACE, f"{material_id}:{index}:{content_hash}")
43          metadata = document.metadata
```

splitter 是递归字符分块器，split_documents 保留每个 Document 的 metadata。

separators 从段落到换行、句号、空格逐级尝试，最后空字符串允许逐字符切开。

index 是切分结果序号；content 经过 strip；哈希和 uuid5 随后给片段建立身份。



图 2-5 递归分块的概念流程。真实实现还会按大小合并短单元，重叠量不是每对块都精确相等。

参数 / 配置	本项目设置	改变以后应观察什么
chunk_size	默认 800 字符	变小会增加块数，可能切断完整说明
chunk_overlap	默认目标 120 字符	增加会扩大冗余；实际边界受分隔单元影响
separators	双换行、换行、。！？、.、空格、空串	不同标点会影响边界，中文句号优先于硬切
add_start_index	True	框架分块 metadata 会产生起始位置
length_function	未覆盖，使用字符长度	不是按模型 tokenizer 测量

表 2-5 这是源码默认值；本地 .env 可以覆盖大小与重叠，实验明确传值以保证可复现。

分块器逐个处理 Document，并不会自动把 PDF 第 3 页末尾和第 4 页开头拼成一个知识单元。因此跨页定义可能落在不同块里。减小块大小也不会必然提高召回效果：你需要同时检查主题集中程度、语义完整程度和后续引用可读性。真正的参数选择要用代表性问题和来源标注验证。

官方概念与项目参数

LangChain 官方递归切分说明确认：大小由 length_function 决定，chunk_overlap 是目标重叠。本文的字符单位、中文分隔符和 800/120 取自当前项目源码。

2.6 片段身份与元数据：重复处理怎样保持可追溯？

同一份资料重新处理，如果文本和分块顺序没有变化，系统希望复用已有片段身份。项目先对规范化后的正文计算 SHA-256，再用固定命名空间、材料 ID、序号与哈希生成 UUID5。UUID5 在同样输入下会得到同样结果，因此重复处理能够与已有数据库记录对应起来。

源码节选：把框架分块转成领域对象

定位：apps/api/study_agent/ingestion/chunking.py，第 44-61 行。

```
        chunks.append(
            DocumentChunk(
                id=chunk_id,
                material_id=material_id,
                chunk_index=index,
                content=content,
                token_count=_estimate_tokens(content),
                page_start=metadata.get("page_start"),
                page_end=metadata.get("page_end"),
                heading_path=list(metadata.get("heading_path", [])),
                content_hash=content_hash,
                vector_id=None,
                embedding_model=None,
            )
        )
    if not chunks:
        raise ValueError("document produced no chunks")
    return chunks
```

DocumentChunk 是 dataclass 领域对象，不等同于 LangChain Document。

page_start、page_end、heading_path 从 metadata 拷贝；向量相关字段初始为 None。

没有有效片段时抛出 ValueError，而不是悄悄把空资料标记成可以学习。

文件的 `sha256` 用于判断上传内容是否重复；片段的 `content_hash` 用于表示某段正文的版本。前者针对原始文件字节，后者针对切分后的字符串，它们不能互换。PDF 的排版变化可能改变文件哈希，却不改变提取出的文字；修改片段正文则会改变内容哈希并影响片段 ID。

字段 / 方法	输入与含义	不能据此推出什么
material_id	当前资料的 UUID	相同文字放在不同资料里不会拥有同一片段 ID
chunk_index	当前分块结果的顺序	不是页码；插入前部块可能影响后续 ID
content_hash	正文 UTF-8 字节的 SHA-256	不包含标题路径与页码
uuid5	命名空间 + 材料 ID + 序号 + 哈希	不是全局按相同文本去重
token_count	英文数字词串数 + 中文字符数，最低 1	不等于真实模型 Token 使用量

表 2-6 身份、顺序、版本与计量的对照。每个字段回答一个不同问题。

框架生成的 `start_index` 没有进入当前 DocumentChunk 模型，`source` 文件名也没有直接写入这个对象。业务链路主要通过 material_id 找回资料，再使用页码和标题定位。PDF 里读到某个 metadata 字段时，应继续追踪它是否被持久化、是否被工具返回；不能假定所有 metadata 都贯穿全程。

2.7 重新处理与失效：为什么更新材料会影响题目？

题目会引用材料片段，辅导回答也会引用片段证据。当资料重新解析后，系统比较旧片段 ID 与当前片段 ID，找出过期片段。过期来源需要触发题目来源失效，并清理相关向量。这种关联解释了为什么“重新处理”不是简单再上传一遍，也说明了稳定 ID 对下游引用的重要性。



图 2-6 当前重新处理的关键关系。这里只概括依赖顺序，不能把跨数据库操作看作一个原子事务。

源码节选：识别过期片段与关联题目

定位：apps/api/study_agent/ingestion/tasks.py，第 125-139 行。

```
125         current_ids = {chunk.id for chunk in chunks}
126         stale_ids = set(existing_models) - current_ids
127         orphaned = await SqlAlchemyQuestionRepository(session).invalidate_sources(list(stale_ids))
128         if orphaned:
129             await get_question_index().delete_questions([question.id for question in orphaned])
130             for question in orphaned:
131                 model_question = await session.get(QuestionModel, question.id)
132                 if model_question:
133                     model_question.vector_id = None
134         # Delete obsolete indexes first: new content may reuse a chunk_index.
135         if stale_ids:
136             await session.execute(
137                 delete(DocumentChunkModel).where(DocumentChunkModel.id.in_(stale_ids))
138             )
139             await session.flush()
```

existing_models 来自当前数据库，chunks 来自新一轮解析和分块。

stale_ids 是旧 ID 集合减去当前 ID 集合；invalidate_sources 处理旧题目引用。

旧向量和旧片段会清理；这段代码运行在后台任务上下文，不能单独复制执行。

数据库、对象存储和 Milvus 是不同组件，一次数据库提交不能把所有外部写入同时回滚。阅读处理代码时，应关注失败后哪些阶段已经完成、再次运行是否复用稳定 ID、旧索引是否被删除。对本章而言，先能画出这些依赖，再到工程验证章节研究具体故障与恢复条件。

ready 的实际含义

ready 表示当前处理链路已走完，允许业务继续使用资料。它不保证 OCR 质量、语义切分质量或全文出题覆盖，后面仍需专项检查。

2.8 完整离线实验：自己观察标题、块边界和稳定 ID

在项目根目录新建自己的 `ch02_parse.py`，复制下面完整代码，使用 `apps/api/.venv/Scripts/python.exe` 运行。实验直接调用生产解析与分块函数，输入是固定 TCP Markdown；不读取你的飞书文档、不写业务数据库，也不调用外部模型。把块大小设为 24，是为了让短文本也出现明显边界。

完整可运行实验：解析、分块与身份比较

```python
from uuid import UUID

from study_agent.ingestion.chunking import build_chunks
from study_agent.ingestion.parsers import parse_document

text = (
    "# 网络\nTCP 提供可靠传输。\n\n"
    "## 三次握手\n"
    "客户端发送 SYN，服务器回复 SYN+ACK，客户端回复 ACK。"
)
documents = parse_document(text.encode("utf-8"), "网络.md")
options = dict(material_id=UUID(int=1), chunk_size=24, chunk_overlap=6)
chunks = build_chunks(documents, **options)
again = build_chunks(documents, **options)
other = build_chunks(documents, **dict(options, material_id=UUID(int=2)))
assert [c.id for c in chunks] == [c.id for c in again]
assert chunks[0].id != other[0].id
print("documents:", len(documents))
print("heading:", documents[1].metadata["heading_path"])
print("chunks:", len(chunks))
print("stable_ids:", True)
print("same_text_other_material_same_id:", False)
for chunk in chunks:
    print(chunk.chunk_index, len(chunk.content),
          chunk.token_count, chunk.heading_path, repr(chunk.content))

try:
    parse_document(b"   ", "empty.txt")
except ValueError as exc:
    print("empty_error:", str(exc))
```

encode 把字符串变为文件字节；parse_document 依据 .md 后缀解析。

options 固定材料 UUID、字符大小和目标重叠；第二次运行用于比较 ID。

改用另一 material_id 比较相同文本身份；最后使用空文本观察失败信息。

结果解释：本次执行得到 documents=2、chunks=4、stable_ids=True；第二段标题路径为 ['网络','三次握手']；空文件错误为 document contains no extractable text。

4 个块的字符长度依次是 11、15、17、1，对应粗略 token_count 为 7、11、8、1。最后的“。”被单独保留下来，说明当前清洗只排除空字符串，没有过滤纯标点片段。这个人工设置的小块参数揭示边界行为；不能据此宣称默认 800 字符下所有真实资料都存在同样问题。

实验还证明：同一材料、相同文本和序号重复运行，片段 ID 一致；换一个材料 ID 后，相同文本也产生不同 ID。它没有验证 PDF 阅读顺序、Milvus 服务或模型答案。继续学习时，应把“本次代码实际输出”与“系统端到端效果”分开记录，让每个结论都有明确的验证范围。

- 把 chunk_size 改为 40，逐块比较哪些句子重新合并。
- 把 chunk_overlap 改为 0，检查块边界是否失去必要上下文。
- 保留正文，只改标题名称，比较 heading_path 与 content_hash。
- 输入只有空白的 Markdown，再输入 .csv 文件，区分空文本和后缀不支持两类错误。

2.9 本章回顾：分块完成，为什么还不能保证全文覆盖?

把全文切成块，只说明可提取文字进入了候选数据；实际哪些块进入模型，是另一层选择。当前出题最多取前、中、后三组，每组最多四块，标题、分隔符和正文都受总字符预算约束。它扩大了分布范围，但仍是抽样，不代表通读整份资料，也没有查询式向量召回。

已有 `examples/learning/rag_context.py` 使用 64 页固定文本，直接调用生产分块与出题上下文函数。2026-10-07 的学习准备记录中，24,000 字符预算选入前 37 个片段，第 64 页没有进入输入。这里引用的是此前记录，数字不代表本章新增运行，也不代表所有资料都会选入相同块数。

> **长资料学习的关键问题**
>
> 资料 ready 与全文覆盖不是同一件事。未来可评估分批生成、章节配额和批次去重；这些属于明确扩展，不把当前系统写成已经保证完整覆盖。

复盘时，用一段话说明你亲自观察过的输入、分块参数、输出和限制。比如：通过固定 Markdown 验证标题路径与重复处理身份，发现小块参数会产生纯标点片段；随后提出按结构或内容规则过滤的方案，并说明需要哪些真实材料来验证。这样的记录比只背“递归切分保留语义”更有价值。

完整可运行实验：64 页材料的实际输入范围

```python
import runpy
from pathlib import Path

module = runpy.run_path(str(Path.cwd() / "examples/learning/rag_context.py"))
result = module["run_demo"]()
print("total_chunks =", result["total_chunks"])
print("group_targets =", result["group_targets"])
print("selected_pages =", result["selected_pages"])
print("context_chars =", result["context_chars"])
print("last_page_included =", result["last_page_included"])
assert result["context_chars"] <= result["configured_max_chars"]
assert result["selected_chunks"] < result["total_chunks"]
```

在项目根目录用默认 .venv 运行；输入是固定教学文本，不调用模型、向量服务或数据库。

三组目标题数为 4 / 3 / 3，选中第 1-4、31-34、61-64 页；没有被选中的页依然是覆盖缺口。

结果解释：64 块中选取 12 块，context_chars = 7206，第 64 页进入输入；这证明分组抽样行为，没有证明全章节覆盖。

2.10 综合练习与参考思路

练习 1：默认 chunk_size=800 为什么不能说成 800 Token?

参考思路：当前分块器按字符长度切分；token_count 是项目启发式估算，真实 tokenizer 未参与块大小计算。

练习 2：扫描 PDF 为什么可能无法解析?

参考思路：当前使用 pypdf 提取文字，没有 OCR；若全部页面无可提取文字，parse_document 会拒绝空结果。

练习 3：标题路径改变，片段 ID 一定改变吗?

参考思路：不一定。ID 使用材料 ID、块序号和正文哈希，不直接包含标题或页码，须分别观察元数据。

练习 4：全文已经切块，为什么出题仍可能遗漏某些章节?

参考思路：分块结果完整不等于输入覆盖完整。当前最多取三组、每组四块，字符预算还可能截断正文；中间未选窗口依然可能遗漏。

2.11 源码阅读清单与下一步

文件或入口	关注的函数 / 类	阅读时回答的问题
apps/api/study_agent/ingestion/parsers.py	parse_document / _parse_markdown / _parse_pdf	每种文件输入如何变成带 metadata 的 Document?
apps/api/study_agent/ingestion/chunking.py	build_chunks / _estimate_tokens	大小的单位是什么，ID 由哪些字段决定?
apps/api/study_agent/domain/models.py	DocumentChunk	框架 metadata 中哪些字段进入业务对象?
apps/api/study_agent/ingestion/tasks.py	parse_material / index_material / finish_ingestion	资料状态、过期来源和索引怎样协作?
apps/api/study_agent/ingestion/dispatcher.py	CeleryIngestionDispatcher.dispatch	三个后台任务以什么顺序执行?
examples/learning/rag_context.py	run_demo	分块与模型上下文的覆盖范围为什么不同?
apps/api/study_agent/question_generation/context.py	plan_generation_groups	窗口抽样、题数分配与真实输入截断怎样协作

下一章：第 3 章把这些带身份与来源的片段转成向量，追踪检索候选如何经过 PostgreSQL 校验，最终成为可引用证据。

主要参考：本地飞书 LangChain、Agent、RAG 与 LangGraph 课程的教学组织方式，以及当前项目源码。具体框架接口以锁定环境与官方文档为准。